

ShockProof — Supply Chain Shock Simulator

Complete A–Z Interview Prep Guide — architecture, algorithms, code, and rehearsed Q&A, compiled from a full read-through of the codebase.

30-second pitch: ShockProof is a full-stack supply-chain risk analytics platform I built end-to-end: a PostgreSQL data model of 100 suppliers / 500 products / their sourcing relationships and 2 years of transaction history, synthetic data statistically calibrated against a real public logistics dataset (DataCo), a NetworkX-based dependency graph with PageRank/centrality analysis, a vectorized NumPy Monte Carlo engine that simulates 5 disruption scenarios (port strikes, factory shutdowns, etc.) across every supplier to produce P50/P95 financial-loss distributions, a composite multi-factor resilience scoring model, an ROI-driven mitigation playbook generator, a FastAPI backend with ~15 endpoints, a React 19 dashboard frontend, a 45-test pytest suite, and — the part I'm proudest of — a rigorous statistical evaluation framework that benchmarked my hand-tuned scoring model against the simulated ground truth, found it underperforming, diagnosed why with a noise-ceiling analysis, and fixed it with a disjoint train/test grid-search weight optimization.

Table of Contents

1. Business Problem & Framing
2. System Architecture
3. Tech Stack
4. Data Model (PostgreSQL Schema)
5. Synthetic Data Generation & Real-World Calibration
6. Notebook Pipeline (00–05)
7. Core Module: db.py
8. Core Module: graph.py (Graph Theory)
9. Core Module: simulation.py (Monte Carlo)
10. Core Module: scoring.py (Resilience Scoring)
11. Core Module: scenarios.py
12. Core Module: playbook.py (ROI Engine)
13. API Layer (FastAPI)
14. Frontend (React App)
15. Design System & Mockups
16. Evaluation Framework (★ centerpiece)
17. Test Suite
18. Root-Level Scripts & Dev Artifacts
19. Known Inconsistencies (self-aware talking points)
20. Project History (git phases)
21. Rehearsed Interview Q&A
22. Formula & Number Cheat Sheet

1. Business Problem & Framing

Global manufacturers depend on hundreds of suppliers, many single-sourced or geographically clustered. A single disruption (port strike, factory fire, currency shock) can cascade into major revenue loss, but most companies only find out how exposed a supplier is *after* the disruption hits. ShockProof answers three questions proactively for a procurement/risk team:

- 1. **Where are we structurally fragile?** (graph analysis — sole-source and highly-central suppliers)
- 2. **How bad could it get, in dollars, and how likely?** (Monte Carlo simulation of 5 disruption types → P50/P95 loss distributions)
- 3. **What should we do about it, and is it worth the money?** (composite risk scoring → priority matrix → costed mitigation playbook with ROI/payback period)

The project name is **ShockProof** (see `docker-compose.yml` container name `shockproof_db` and the app's `<title>`). Domain context is an **India-based sporting-goods manufacturer** (currency formatted in ₹ Lakh/Crore in the UI; product categories are Cleats, Cardio Equipment, Camping & Hiking, Fishing, Indoor/Outdoor Games, Men's Footwear, Water Sports, Women's Apparel).

2. System Architecture

Four layers, cleanly separated:

Layer	Contents	Responsibility
Data	<code>data/schema.sql</code> , <code>data/generate_synthetic_data.py</code> , PostgreSQL	7-table relational schema; synthetic but empirically-calibrated data generator
Analytics core	<code>src/graph.py</code> , <code>simulation.py</code> , <code>scoring.py</code> , <code>scenarios.py</code> , <code>playbook.py</code>	Pure pandas/numpy/networkx functions — DataFrame-in, DataFrame-out, DB-agnostic logic
Data access	<code>src/db.py</code>	SQLAlchemy engine singleton, read/write/query helpers
API	<code>api/main.py</code> (FastAPI)	Glues DB + analytics together, serves JSON to the frontend, ~15 REST endpoints
Frontend	<code>app/</code> (React 19 + Vite)	9-page executive dashboard consuming the API
Evaluation	<code>evaluation/</code>	Independent statistical layer that benchmarks/validates/re-fits the scoring model

Key architectural decision: the analytics functions are **stateless** and take/return DataFrames, so they can run either as an offline batch ETL (writing to precomputed result tables) or be **recomputed live inside API request handlers**. In practice, most "read" endpoints in `api/main.py` actually re-run the Monte Carlo simulation / playbook generation / centrality analysis *live on every request*, using the precomputed DB tables only as a fallback if the live computation fails. This is a deliberate "always-fresh data" trade-off over

latency — worth being ready to discuss the scalability implications (no caching layer, no background job queue, no memoization; `/api/simulation/{scenario_id}` alone runs 100 suppliers × 10,000 Monte Carlo draws per call).

3. Tech Stack

Area	Technology
Backend language	Python (pandas, numpy, scipy, networkx, sqlalchemy, psycopg2-binary)
API framework	FastAPI + Uvicorn, Pydantic response models
Database	PostgreSQL 15 (via Docker Compose)
Synthetic data	Faker, numpy random (seeded, reproducible)
Notebooks	Jupyter, matplotlib, seaborn, plotly
Testing	pytest (45 tests, mix of unittest.TestCase and plain pytest)
Frontend	React 19, React Router v7, Vite 8, Recharts, react-force-graph-2d (canvas)
Frontend styling	Hand-written CSS + custom properties (design tokens), no Tailwind/CSS-in-JS in production app
Frontend linting	oxlint (Rust-based)
Containerization	Docker Compose (Postgres only — API/frontend run natively)

4. Data Model (PostgreSQL Schema)

`data/schema.sql` defines 7 tables, split into core/source tables and computed-results tables.

Core tables

Table	Key columns	Notes
suppliers	supplier_id PK, supplier_name , country , tier , reliability_score , avg_delay_days , delay_volatility , rejection_rate	tier CHECK (tier IN (1,2)) — Tier-1/Tier-2 supplier classification
products	product_id PK, sku UNIQUE, category , unit_cost , monthly_demand , safety_stock_days	
supply_relationships	relationship_id PK, FKs to suppliers/products (ON DELETE CASCADE), supply_share , is_sole_source	CHECK (supply_share BETWEEN 0 AND 1) ; UNIQUE(supplier_id, product_id)
transactions	transaction_id BIGSERIAL PK, order/promised/actual dates, quantities ordered/delivered/rejected	BIGSERIAL anticipates high row volume (730 days × many relationships)

Computed-results tables (populated by the pipeline, read/cached by the API)

Table	Key columns
simulation_results	per supplier×scenario: p50_impact , p95_impact , run_count
resilience_scores	dependency_risk , geo_risk , reliability_risk , substitutability_risk , composite_score
mitigation_playbook	recommended_action , estimated_cost , p95_impact , roi
risk_priority_matrix	supplier_id PK (1:1 — single current snapshot), probability_tier , impact_tier , priority_quadrant

Indexes on all FK/filter columns (`transactions.supplier_id/product_id/order_date` ,
`supply_relationships.supplier_id/product_id`). All result-table FKs cascade-delete with their parent supplier.

5. Synthetic Data Generation & Real-World Calibration

`data/generate_synthetic_data.py` — reproducible (`np.random.seed(42)`) generator for 100 suppliers
/ 500 products. **Not arbitrary data** — constants (delay mean/std, category late-delivery rates, country

market shares, monthly seasonality, cost log-normal parameters) are lifted from statistical analysis of the real, public **DataCo Global Supply Chain dataset** (`data/raw/DataCoSupplyChainDataset.csv`), fitted in notebook `00_data_calibration.ipynb` via MLE + Kolmogorov–Smirnov goodness-of-fit testing.

Deliberately "seeded" risk scenarios (so the algorithms have known ground truth to catch)

- **Supplier 1** — hardcoded sole-source fastener supplier (71% share of the "Cleats" category, `is_sole_source=True`) — a textbook single point of failure.
- **Suppliers 2, 3, 4** — hardcoded co-located China cluster, splitting 70% of "Cardio Equipment" via a Dirichlet draw — designed to trigger the geographic-concentration risk logic.
- Remaining 96 suppliers get countries via weighted random choice from a 10-country DataCo-calibrated distribution.

Key distributions used

Quantity	Distribution	Why
Unit cost	Log-Normal ($\mu=4.4617$, $\sigma=0.8595$)	Cost data is strictly positive, right-skewed
Monthly demand	Poisson, category-specific λ	Count data
Supply share (most categories)	Dirichlet(1.5,...,1.5) among 2–4 suppliers	Shares sum to 1 by construction
Delivery delay	Normal($\mu = \text{DELAY_MEAN} \times \text{tier_factor}$, $\sigma = \text{DELAY_STD}$)	Tier-1 suppliers get 0.8× the delay (more reliable)
Order quantity (daily)	Poisson around seasonally-adjusted expected demand	
Rejections	Binomial($n=\text{qty_ordered}$, $p=\text{category late-rate} \times 0.05$)	

Generator drops/recreates all 7 tables, re-syncs PostgreSQL SERIAL sequences after bulk insert (necessary because pandas `to_sql` bypasses sequences), writes CSV backups to `data/raw/synthetic/`, and prints a data-quality report (verifies supply shares sum to 1.0 ± 0.001 per product).

6. Notebook Pipeline (00–05)

A literate, "Theory → Code → Interpretation" notebook sequence in `notebooks/` that tells the project's end-to-end data-science story, later mirrored/productionized into `src/` and `api/`.

00_data_calibration.ipynb

Fits real DataCo data: Normal vs Gamma MLE fit to shipping delays (winner picked via KS statistic), late-delivery rate per product category, Pareto/geographic concentration of order volume by country, monthly demand seasonality multipliers. Output feeds the synthetic generator's constants.

01_etl_pipeline.ipynb

Pulls suppliers/products/relationships/transactions from Postgres, runs explicit PASS/FAIL data-quality gates (supply shares sum to 1, referential integrity, chronological validity), computes `avg_delay_days`, `delay_volatility`, `rejection_rate`, and the canonical **reliability score**:

```
reliability_score = 1.0 - (0.4·norm(avg_delay) + 0.4·norm(delay_volatility) +  
0.2·norm(rejection_rate))
```

Note: this 0.4/0.4/0.2 formula differs from `src/scoring.py`'s `compute_reliability_risk()`, which uses 0.5/0.5 on delay/volatility only (no rejection term) — a real inconsistency, see §19.

02_graph_construction.ipynb

Models the chain as a directed bipartite graph (Supplier→Product). Documents an important correction: naive PageRank on the raw graph gives every supplier the same baseline score (suppliers have zero in-degree), so PageRank is run on the **reversed graph** (Product→Supplier) so product importance (driven by demand/cost) flows back to the suppliers serving it. Also computes degree/betweenness/closeness centrality and cross-references high-centrality + low-reliability suppliers as the most dangerous single points of failure.

03_monte_carlo_engine.ipynb

The stochastic simulation engine — same logic as `src/simulation.py` (below). Runs 100 suppliers × 5 scenarios × 10,000 draws = 5,000,000 simulated trials, producing P50/P95 heatmaps, radar comparisons across scenarios, and the "worst case scenario per supplier" analysis.

04_resilience_scoring.ipynb

Builds and self-validates the composite resilience score. Checks factor collinearity (all pairwise correlations < 0.6, so treated as independent dimensions safe to linearly combine), decomposes scores into weighted factor contributions per supplier, runs a 5-config sensitivity analysis, and validates against simulation via **Pearson correlation** (reports a strong significant negative correlation with P95 exposure). **This Pearson-based self-validation is in tension with the evaluation harness's later Spearman rank-correlation finding that the same composite score performs worse than random — see §16 and §19, a genuinely**

interesting metric-choice story. Frames the priority matrix explicitly as **ISO 31000**-aligned (separating Likelihood from Consequence into a quadrant instead of one blended number).

05_business_outputs.ipynb (+ .py twin)

The capstone/executive notebook — turns scores + simulation + priority matrix into the costed, ROI-ranked mitigation playbook, per-supplier "executive briefing" cards, cost-of-action-vs-cost-of-inaction charts, and a projected network-wide "before vs after mitigation" resilience number.

7. Core Module: `src/db.py`

Thin SQLAlchemy wrapper: lazy singleton `get_engine()` builds a `postgresql://` URL from env vars (`DB_HOST/PORT/NAME/USER/PASSWORD`, via `python-dotenv`). `get_connection()` is a context manager that wraps `SQLAlchemyError` into `RuntimeError`. `read_table / read_query` wrap `pd.read_sql_table / pd.read_sql`; `write_dataframe` wraps `to_sql` with **default** `if_exists='replace'` (destructive — re-running ETL drops and recreates tables, worth flagging as a gotcha). `execute_statement` uses `engine.begin()` for auto-commit transactions.

8. Core Module: `src/graph.py` — Graph Theory

Business question: which suppliers are structurally critical — if they vanished, how much of the product catalog is affected? Uses **NetworkX** `DiGraph` .

`build_dependency_graph()`

Builds a **weighted, directed, bipartite** graph. Nodes prefixed `S_{id}` / `P_{id}` to avoid collisions. Supplier nodes carry `country` , `tier` , `reliability_score` ; edges go Supplier→Product with `weight = supply_share` .

`compute_pagerank(G, alpha=0.85)`

Standard PageRank via power iteration, damping factor $\alpha = 0.85$ (Google's canonical default). Returns `node_id/node_type/pagerank_score` sorted descending; scores sum to ≈ 1.0 (probability distribution property).

$$PR(u) = (1-\alpha)/N + \alpha \cdot \sum_{v \in \text{In}(u)} PR(v)/\text{Out}(v)$$

`compute centrality_metrics(G)`

Degree, betweenness, closeness centrality for supplier nodes, computed on the raw bipartite `DiGraph`.

Subtlety worth owning in interview: betweenness/closeness are structurally degenerate on a strict bipartite graph (suppliers never connect directly to each other), so this function's numbers are less meaningful than they look. The API's `/api/centrality` endpoint fixes this correctly by first **projecting** onto a supplier-only graph via `networkx.algorithms.bipartite.weighted_projected_graph` (two suppliers get an edge if they co-supply ≥ 1 product) before computing betweenness/closeness — the single most sophisticated piece of graph theory in the codebase. Two different implementations of "centrality" coexist: know both and be ready to explain why the API's version is the correct one.

`identify_critical_suppliers()`

Flags suppliers that are (a) sole-source for any product, or (b) in the top-20% by PageRank (`quantile(0.80)`). Merges with centrality metrics, returns top N (default 10) by PageRank.

9. Core Module: `src/simulation.py` — Monte Carlo Engine

Business question: for a given supplier and disruption type, what's the *distribution* of possible financial impact — not just a point estimate, since rare fat-tailed events are exactly what a single average would hide?

Approach: fully **vectorized** NumPy Monte Carlo — `n_runs=10,000` draws per supplier×scenario, no Python loop in the hot path.

estimate_beta_params(a, c) — Method of Moments / PERT

```
mean = (a+c)/2 std = (c-a)/6 (PERT heuristic: min/max span  $\approx 6\sigma$ )
```

Solves standard method-of-moments formulas for Beta(α , β); falls back to `$\alpha=\beta=4.0$` (symmetric bell-shaped Beta) if the variance constraint `var < mean(1-mean)` is violated or if `a==c`.

run_simulation(supplier, scenario, ..., n_runs=10000) — the core routine

1. **Modulate annual probability** by supplier attributes:

- `reliability_modulated` scenarios: `prob *= (1 + norm(delay_volatility))`
- `rejection_modulated` scenarios: `prob *= (1 + norm(rejection_rate))`
- `geographic_modifier` scenarios: $\times 1.5$ if supplier's country $\in \{\text{China, US, Mexico, Brazil}\}$ (coastal/high port-exposure countries)

clamped to [0,1].

2. **Draw random variables** (vectorized over `n_runs`): shock occurrence $\sim \text{Bernoulli}(\text{prob})$; duration $\sim \text{Uniform}(\text{min}, \text{max})$ days; supply-loss fraction $\sim \text{Beta}(\alpha, \beta)$ scaled into `[loss_min, loss_max]`.

3. **Revenue-at-risk weight:** join relationships→products, `weighted_revenue = supply_share × unit_cost × monthly_demand`, summed per supplier.

4. **Vectorized impact formula:**

```
revenue_at_risk = loss_fraction × (duration_days / 30.0) ×  
total_product_revenue_weight  
revenue_at_risk = where(shock_occurred,  
revenue_at_risk, 0) ← zero-inflated, right-skewed
```

Extracts `p50_impact`, `p95_impact`, `mean_impact`, `std_impact`, `zero_impact_fraction`, and returns the **raw 10,000-length array** too, for downstream stats.

analyse_distribution()

Mean/median/std/percentiles (p50/p75/p90/p95/p99), skewness & kurtosis (via `scipy`, with a manual fallback formula), and `var_ratio = p95/mean` — a simple tail-fatness metric (how many multiples of average loss the P95 tail represents).

identify_worst_case_scenarios()

`total_p95_exposure` = **sum** of P95 impact across all 5 scenarios (additive/conservative aggregate, not a single worst case); `worst_case_scenario` = the single scenario with max P95 for that supplier. Feeds directly into scoring's priority matrix and the playbook.

10. Core Module: `src/scoring.py` — Resilience Scoring

Four independent risk sub-scores, each min-max normalized to [0,1], combined with fixed weights into one composite resilience score.

Factor	Weight	Formula
Dependency risk	0.40	Max supply_share per supplier, clipped at 0.50 (risk plateaus above 50% share)
Geographic risk	0.25	Fraction of suppliers co-located in same country; +0.15 flat penalty if country has ≥ 3 suppliers (cluster-threshold effect, not linear)
Reliability risk	0.20	$0.5 \cdot \text{norm}(\text{avg_delay_days}) + 0.5 \cdot \text{norm}(\text{delay_volatility})$
Substitutability risk	0.15	Inverse of avg. alternative-supplier count per product; sole-source suppliers hard-coded to exactly 1.0

```
composite_risk = 0.40·Dep + 0.25·Geo + 0.20·Rel + 0.15·Sub
resilience_score = 1.0 - composite_risk (higher = safer)
```

Risk bands — two DIFFERENT schemes coexist (know both!)

Scheme	Used in	Logic
Quartile-based (relative)	<code>compute_resilience_scores()</code> — the actual production banding	$\leq Q25$ Critical, $Q25-Q50$ High, $Q50-Q75$ Medium, $>Q75$ Low — always exactly ~25% "Critical" regardless of absolute risk level
Absolute thresholds	<code>get_score_interpretation()</code> — narrative/interpretation text	<0.30 Critical, $0.30-0.50$ High, $0.50-0.70$ Medium, >0.70 Low

`sensitivity_analysis()`

Recomputes scores under 5 weight configs (default + 4 "70% on one factor" configs) to test how robust supplier risk-band classification is to weighting assumptions — a standard risk-model robustness check, and the direct predecessor to `evaluation/learn_weights.py`'s more rigorous version.

`compute_priority_matrix()` — probability × impact risk matrix (ISO 31000 style)

Probability tier from resilience-score quartiles (inverted); impact tier from `total_p95_exposure` quartiles. Quadrant logic (note precedence — probability checked before impact):

Condition	Quadrant
High Probability + High Impact	Critical Priority
High Probability (any impact)	Monitor Closely
High Impact only	Contingency Plan
else	Routine Review

11. Core Module: `src/scenarios.py`

`DisruptionScenario` is a `@dataclass` with 4 boolean modifier flags. Five predefined scenarios (exact constants):

Scenario	Annual prob.	Duration (days)	Supply loss	Modifier
Port Strike	0.18	14–45	0.60–1.00	<code>geographic_modifier</code>
Factory Shutdown	0.12	7–30	0.80–1.00	<code>reliability_modulated</code>
Currency Shock	0.12	30–90	0.20–0.50	<code>affects_all_in_country</code> (declared but never actually consumed in <code>simulation.py</code> – latent/unused flag)
Logistics Delay	0.35	5–21	0.30–0.70	none (baseline, highest annual probability)
Quality Failure	0.10	14–60	0.40–0.90	<code>rejection_modulated</code>

12. Core Module: `src/playbook.py` — Mitigation & ROI Engine

Business problem: turn "this supplier is risky" into "here is the specific action, its cost, and whether it's worth the money" — the piece that makes this a decision-support tool, not just a dashboard.

5-action catalogue (MITIGATION_ACTIONS)

Action	Cost	Risk reduction
Dual-Sourcing	8% of annual revenue exposure	50% dependency, 60% substitutability
Safety Stock Increase	(extra days/30) × monthly revenue × 5%; 30 days if Critical band, 15 if High	25% dependency
Supplier Development	flat \$200,000	40% reliability
Geographic Diversification	12% of annual revenue exposure	45% geographic
Quarterly Monitoring	flat \$15,000	15% reliability (low-cost default)

`recommend_action()` — decision tree by priority quadrant

Quadrant	Rule
Critical Priority	substitutability ≥ geographic → Dual-Sourcing, else Geographic Diversification
Monitor Closely	dependency ≥ reliability → Safety Stock Increase, else Supplier Development
Contingency Plan	always Dual-Sourcing (comment: "high impact justifies cost despite lower probability")
Routine Review	always Quarterly Monitoring

`compute_roi()`

```
expected_annual_loss = total_p95_exposure × scenario_probability
risk_reduction = expected_annual_loss × reduction_pct(action)
roi = risk_reduction / action_cost
(benefit-cost ratio, not IRR)
payback_period_years = action_cost / risk_reduction
```

Worth being able to critique: using P95 (a tail statistic) as if it were an expected value when multiplying by probability is not a strict expected-value calculation — it's a deliberately *conservative* proxy. Good to acknowledge if asked "is this ROI number rigorous?"

`generate_playbook()` sorts the final table by ROI descending ("best bang-for-buck first"). **Its risk-band fallback logic uses yet a *third* threshold set (0.15/0.40/0.70) — different again from both schemes in `scoring.py`.** `playbook_summary()` computes portfolio budget/ROI only over the "at-risk" cohort (Critical Priority + Monitor Closely), excluding Routine Review/Contingency from financial totals but still counting them in action distribution.

13. API Layer (FastAPI) — `api/main.py`

CORS wide open (`allow_origins=["*"]` , explicitly commented "development — lock down in production"). `lifespan` pings DB on startup without crashing on failure. Helper `_safe()` converts numpy/pandas scalars + NaN for JSON safety.

"Stochastic wiggle" pattern: many endpoints inject small random noise (e.g. `np.random.uniform(0.985, 1.015)` multiplicative on exposure figures) into DB-read values before returning them, purely so the dashboard *feels* live on refresh — a UX/demo trick, not a statistical signal. Comments in the code call this out explicitly ("simulate dynamic updates").

Endpoint	What it does
GET <code>/api/suppliers</code>	All suppliers + resilience scores + priority quadrant; filters: <code>risk_band</code> , <code>priority_quadrant</code>
GET <code>/api/suppliers/{id}</code>	Full profile: risk decomposition, interpretation text, live-recomputed simulation for all 5 scenarios, playbook entry. 404 if missing.
GET <code>/api/simulation/{scenario_id}</code>	Live Monte Carlo for all suppliers under one scenario (100 × 10,000 draws per call), sorted by P95 desc
GET <code>/api/scenarios</code>	All 5 scenario definitions
GET <code>/api/priority-matrix</code>	Full <code>risk_priority_matrix</code> table
GET <code>/api/playbook</code>	Live-generates the mitigation playbook end-to-end; filter: <code>priority_quadrant</code>
GET <code>/api/kpis</code>	Executive summary: total exposure, critical/monitor counts, budget, projected resilience before/after mitigation
GET <code>/api/graph</code>	Full dependency graph nodes/edges + PageRank, for visualization
GET <code>/api/health</code>	DB health check (<code>SELECT 1</code>)
GET <code>/api/products</code>	All products; filter: <code>category</code>
GET <code>/api/supply-relationships</code>	Enriched with names; filters: <code>supplier_id</code> , <code>product_id</code>
GET <code>/api/transactions</code>	Enriched, sorted desc, <code>limit</code> param
GET <code>/api/critical-suppliers</code>	Precomputed <code>critical_suppliers</code> table
GET <code>/api/centrality</code>	Correctly projects bipartite→supplier-only graph before computing betweenness/closeness; approximate betweenness (<code>k=min(50, n)</code>) for graphs >50 nodes
GET <code>/api/sensitivity</code>	5-weight-config sensitivity table per supplier

14. Frontend (React App) — `app/`

React 19 + Vite 8 + React Router v7. **No global state manager** — every page independently fetches its own data via local `useState / useEffect` (no Redux/Context/React Query — every navigation re-fetches from scratch). Charting: mostly hand-rolled SVG/CSS (histograms, donuts, gauges, matrix scatter) plus Recharts for Treemap/Bar/Pie specifically in `SupplierRisk.jsx`. Network graph via canvas-based `react-force-graph-2d`.

9 pages

Route	Page	What it answers
<code>/</code>	Executive Dashboard	KPI cards, risk register, 2x2 priority matrix scatter, top-3 recommended actions
<code>/network</code>	Network Overview	Interactive force-directed graph, geographic concentration, centrality tables
<code>/suppliers</code>	Supplier Risk	Table / Treemap / Triage (kanban) views, cross-filterable mini-charts
<code>/suppliers/:id</code>	Supplier Detail	4 resilience gauges, simulation bar chart, recommended action, operational metrics
<code>/scenarios</code>	Scenario Analysis	Per-scenario simulation with live auto-refresh (10s countdown), hand-built Monte Carlo histogram (P50/P95 markers)
<code>/playbook</code>	Mitigation Playbook	ROI-ranked accordion, cost/loss chart, action-type donut, 12-month campaign planner (client-side quarter-bucketing algorithm)
<code>/products</code>	Products	Searchable/sortable SKU catalog
<code>/relationships</code>	Relationships	Expandable supplier→product master-detail, sole-source flags
<code>/transactions</code>	Transactions	OTDR/rejection-rate log, delay/rejection filters

Notable frontend design points

- A reusable `useApi()` data-fetching hook exists in `LoadingSpinner.jsx` but is **never actually used** — every one of the 9 pages hand-rolls the same data/loading/error 3-state pattern instead. Good "what would you refactor" answer.
- `api/client.js` hardcodes `BASE_URL = 'http://localhost:8000'` — no env var usage despite Vite natively supporting `import.meta.env`. A real deployment gap.
- Dark/light theme toggle persisted to `localStorage`, applied via a `data-theme` attribute + CSS custom-property overrides.
- Heavy client-side business logic: sole-source detection, dominant-risk-factor computation, delay/rejection-rate derivation, and the entire 12-month campaign-quarter-assignment algorithm all run in the browser from raw API data rather than server-side.
- `App.css` is unused leftover Vite/React starter boilerplate, never cleaned up.

15. Design System & Mockups

`frontend/` holds AI-generated static HTML/Tailwind-CDN mockups (paired `code.html` + `screen.png`) that preceded the real build — a design-exploration phase, not shipped code. Two competing design specs (`titan_resilience_1/DESIGN.md` , `titan_resilience_2/DESIGN.md`) define full YAML color/typography/spacing token systems. **`titan_resilience_1` 's exact tokens (surface `#101418` , primary `#99cbff` , risk colors `#ff6b59` / `#ffa600` / `#464c89` / `#003d5c`) were carried verbatim into `app/src/index.css` as CSS custom properties — a clean, traceable design-tokens→implementation pipeline.** The mockups only covered 5 of the 9 final pages; Supplier Risk, Products, Relationships, and Transactions were built without a prior static mockup. The real network graph (canvas, physics-based, interactive) is a substantial step up from the mockup's static placeholder SVG. `dashboard/app/` is an empty placeholder (`.gitkeep` only).

16. Evaluation Framework (★ centerpiece — best interview material)

Why this is worth leading with: it's a complete, honest scientific narrative — benchmark the hand-tuned model, discover it underperforms even a random baseline, prove statistically that the gap is real (not just noise) via a noise-ceiling analysis, then fix it with a properly held-out grid search, and finally interrogate the fix itself for a subtler flaw. That progression is exactly the kind of rigor interviewers want articulated, especially the willingness to publish a negative initial result.

Step 1 — `evaluation/eval_harness.py` : does the composite score actually rank suppliers correctly?

Ground truth: a fresh Monte Carlo re-run summing P95 exposure across all 5 scenarios. **Cheap methods compared:** composite score, equal-weighted ablation, 4 single-factor ablations (dependency/pagerank/reliability/revenue-weighted only), and a random-shuffle floor.

Methodology: repeated-holdout / bootstrap design — for each of 30 (later 50) seeds, bootstrap-resample transaction history → re-derive reliability features → re-run the full Monte Carlo for that seed's ground truth. Metrics: **Spearman rank correlation**, **Precision@10**, **NDCG@10**, reported as mean ± 95% CI across seeds, with **paired Wilcoxon signed-rank tests** (correct choice for paired, non-normal rank data) comparing composite vs. each baseline.

The finding: the hand-designed composite score scored **Spearman** ≈ -0.047 , **Precision@10** ≈ 0.0 — **worse than the random baseline** (Spearman ≈ 0.016 , **Precision@10** ≈ 0.106) and beaten by a single raw concentration metric (`dependency_only` : Spearman ≈ 0.147). Wilcoxon p-values as low as $1.78e-15$ confirm this is not noise.

Step 2 — `evaluation/ceiling_check.py` : is the task itself just too noisy to solve?

A "noise ceiling" borrowed from psychometrics: how much does ground truth agree with itself across two independent realizations of the same risk process? Two variants: (A) full noise — both features and simulation vary; (B) pure simulation noise — features fixed, only the Monte Carlo seed varies. Compares every ordered pair of independent draws.

Result: **Precision@10** ≈ 0.98 (full) / **0.94** (pure sim noise), **Spearman** ≈ 0.999 for both. The ground truth is highly self-consistent — the task has a *high* ceiling, so the earlier near-zero/negative performance is a real, closeable flaw in the scoring method, not inherent randomness. This licenses the next step rather than "ship as-is, it's unsolvable."

Step 3 — `evaluation/learn_weights.py` : fit the weights to data instead of hand-picking them

Adds a 5th factor, `revenue_weighted` (supply-share concentration weighted by `unit_cost × monthly_demand`). **Strict disjoint train/test split by seed** (20 train, 20 test — no leakage). **Algorithm: exhaustive grid search on the probability simplex** — enumerates all non-negative integer compositions of resolution=25 into 5 parts (step size $1/25=0.04$), vectorized matrix multiply of all candidate weight vectors against the factor matrix per training seed. Grid search chosen over gradient descent because **rank correlation is piecewise-constant and non-differentiable** — a deliberate, well-justified methodological choice given the low (5-D) dimensionality. Objective: maximize mean Spearman on training seeds; best point re-verified with exact tie-corrected `scipy.stats.spearmanr`; evaluated unchanged on held-out test seeds.

The result: the grid search collapsed to **100% weight on `revenue_weighted`**, 0% on all four original factors. On held-out test seeds: Spearman ≈ 0.9993 , Precision@10 ≈ 0.956 — essentially at the noise ceiling.

The nuanced critique (know this cold — it's the best "shows depth" answer in the whole project): because the simulator computes impact as $\text{loss_fraction} \times (\text{duration}/30) \times \Sigma(\text{supply_share} \times \text{unit_cost} \times \text{monthly_demand})$, and `revenue_weighted` is built from almost the same terms, the two are close to *tautologically* correlated. The grid search correctly found the single most informative feature *given how the ground truth is constructed* — but this also exposes a **construct-validity limitation of the simulator itself**: the "expensive" ground truth is largely a re-expression of a feature that's cheap to compute directly. The other four qualitative risk factors (geography, reliability, substitutability) likely matter for risks the simulator doesn't fully capture (correlated regional shocks, reputational risk) even though they contribute little to *this specific* ground-truth metric. This is the difference between "beats the benchmark" and "the benchmark itself has a narrow definition of risk" — a mature thing to say proactively rather than wait to be caught on it.

Result artifacts (evaluation/results/)

Every stage writes both a summary CSV and a raw per-seed/per-pair CSV (`eval_summary.csv` , `eval_significance.csv` , `eval_results_per_seed.csv` , `eval_comparison.png` , `learned_weights.csv` = `[0,0,0,0,1]` , `learned_weights_summary.csv` , `learned_weights_significance.csv` , `learned_weights_test_results.csv` , `ceiling_summary.csv` , `ceiling_full_noise.csv` , `ceiling_pure_simulation_noise.csv`) — good reproducibility hygiene, lets someone re-derive CIs or audit for seed-selection bias without re-running expensive simulations.

17. Test Suite — 45 tests, `pytest -q` → all pass in ~1.5s

File	Style	Highlights
<code>test_graph.py</code>	unittest, 5 tests	PageRank scores sum to ≈ 1.0 ; None-handling for missing <code>reliability_score</code> ; stdout-capture test for <code>graph_summary()</code> 's print output
<code>test_playbook.py</code>	pytest, ~14 tests	Exact dollar-figure ROI verification per action type; DB mocked via <code>unittest.mock.patch</code> ; zero-cost and zero-reduction edge cases; empty-DataFrame edge case
<code>test_scenarios.py</code>	unittest, 4 tests	Spot-checks exact scenario parameters and modifier flags
<code>test_scoring.py</code>	pytest, ~13 tests	Weight-validation error contract; suppliers engineered to hit exactly 1.0/0.0 composite risk; exact boundary values for interpretation thresholds (0.29/0.30/0.50/0.70/0.71); 8-supplier exact-quartile-split test (2/2/2/2)
<code>test_simulation.py</code>	pytest, ~11 tests	P95 $\geq$ P50 invariant; zero-probability scenario forces <code>zero_impact_fraction==1.0</code> exactly; a fully deterministic scenario (<code>prob=1</code> , <code>duration=30</code> , <code>loss=1.0</code>) hand-verifies <code>mean_impact == 5400.0</code> end-to-end with zero randomness

Good general pattern: exact hand-computed assertions wherever the math is simple enough to verify by hand, falling back to invariant-style assertions (bounds, non-negativity, P95 $\geq$ P50) for genuinely stochastic paths where exact values aren't reproducible.

18. Root-Level Scripts & Dev Artifacts

Not part of the analytical pipeline — leftover debugging/one-off scripts from iterative development:

- `test_db.py`, `test_pb.py`, `test_roi.py` — ad-hoc manual smoke-test scripts (not real pytest tests despite the name), used to eyeball one supplier's numbers during development.
- `verify_critical_playbook.py` — a genuine QA script that asserts a business invariant: no Critical-Priority supplier should ever be recommended "Quarterly Monitoring" (the do-nothing action).
- `build_new.py` / `generate_supplier_risk.py` — large string-replacement scripts that patched `SupplierRisk.jsx` programmatically (added Recharts imports, view-toggle buttons, mini-charts) — one-shot code-generation migrations, not imported anywhere.
- `generate_clean.py` — an aborted stub whose own comments narrate a failed edit attempt and recovery via `git checkout`.

19. Known Inconsistencies (self-aware talking points — own these proactively)

1. Three different risk-band threshold schemes exist simultaneously: quartile-based (`scoring.compute_resilience_scores`), absolute 0.30/0.50/0.70 (`scoring.get_score_interpretation`), and a third absolute set 0.15/0.40/0.70 in `playbook.generate_playbook`'s fallback path.

2. Two different reliability-score formulas: ETL notebook uses 0.4/0.4/0.2 (delay/volatility/rejection); `src/scoring.py` uses 0.5/0.5 (delay/volatility only, no rejection term).

3. Two different centrality implementations: `graph.compute centrality_metrics` runs betweenness/closeness on the raw (degenerate) bipartite graph; `api/main.py`'s `/api/centrality` correctly projects to a supplier-only graph first.

4. Pearson vs. Spearman validation tension: notebook 04 self-validates the composite score against P95 exposure using Pearson correlation (reports strong significance); `eval_harness.py` later finds negative *Spearman rank* correlation for the same score. Both can be simultaneously true on skewed data — a small number of extreme-exposure suppliers can dominate a Pearson correlation while the actual rank-ordering is scrambled. Good illustration of why metric choice matters for a ranking task.

5. `affects_all_in_country` flag on Currency Shock scenario is declared but never consumed in `simulation.py` — a latent/unused field.

6. API recomputes analytics live on almost every request (no caching/async job queue/memoization) — a deliberate freshness-over-performance trade-off, worth being ready to discuss how you'd add caching (Redis, background workers, ETag/If-None-Match, or just pre-computed table reads with a scheduled refresh job) if this needed to scale.

7. Frontend has no env-based config (hardcoded `localhost:8000`) and an unused `useApi()` hook duplicated 9× across pages instead of reused.

20. Project History (from git log — 31 commits)

Development proceeded in clear phases, visible directly in commit history: **Phase 0** (project scaffold) → schema/db/synthetic-data-generator → **notebook 00** (calibration) → **notebook 01** (ETL) → dependency-graph module + scaled dataset to 100 suppliers/500 products → **notebook 02** (graph construction, with

an explicit PageRank bugfix commit: "Fix Section 4 PageRank: compute on reversed graph") → disruption scenarios module + tests → **Phases 4B–4G** and **5A–5C** (further backend build-out) → frontend → fix/fix2/fix3/changes (iteration) → evaluation → ceiling value. This maps cleanly onto the "data → graph → simulation → scoring → playbook → frontend → evaluation" narrative used throughout this document.

21. Rehearsed Interview Q&A

Q: Walk me through the system end to end.

A: Synthetic-but-calibrated data (100 suppliers, 500 products, 2 years of transactions) lives in Postgres. `graph.py` builds a bipartite NetworkX graph and runs PageRank/centrality to find structurally critical suppliers. `simulation.py` runs a vectorized 10,000-draw Monte Carlo per supplier×scenario across 5 disruption types to get P50/P95 financial-loss distributions. `scoring.py` combines 4 normalized risk factors into a composite resilience score and a probability×impact priority matrix. `playbook.py` turns each Critical/Monitor supplier into a costed, ROI-ranked mitigation action. FastAPI exposes ~15 endpoints, mostly recomputing live; a React dashboard consumes them across 9 pages. On top of all that, a separate evaluation layer statistically benchmarks whether the hand-tuned scoring model actually agrees with the Monte Carlo ground truth — and I used that to catch and fix a real problem in my own scoring model.

Q: What was the hardest technical problem you solved?

A: The PageRank direction bug in the graph — running PageRank on the raw Supplier→Product graph gave every supplier a near-identical baseline score because suppliers have zero in-degree. The fix was reversing the graph (Product→Supplier) so a product's importance — driven by its demand/cost — flows back to the suppliers serving it. There's a commit specifically fixing this. The second-hardest was later, in evaluation: discovering my hand-tuned composite score performed *worse than random* on rank correlation against simulated ground truth, and having to prove via a noise-ceiling analysis that this wasn't just simulator randomness before I trusted the result enough to act on it.

Q: Why Monte Carlo instead of a simpler point-estimate risk score?

A: Disruption impact is fundamentally a rare, fat-tailed process — a single expected-value number hides the difference between "usually fine, occasionally catastrophic" and "consistently moderate," which need completely different mitigation strategies. Simulating 10,000 draws per supplier×scenario and reporting P50 vs P95 (plus skewness/kurtosis/var_ratio) captures that shape and lets the business reason about tail risk explicitly, which is exactly what a probability×impact risk matrix (ISO 31000-style) needs as its two separate inputs.

Q: How did you validate that your scoring model actually works?

A: I didn't just eyeball it — I built a benchmarking harness that compares my composite score's supplier ranking against the "expensive" ground truth (a fresh Monte Carlo simulation), using Spearman rank correlation, Precision@10, and NDCG@10, aggregated over 30-50 bootstrap seeds with 95% CIs and paired Wilcoxon significance tests against baselines. It found my hand-tuned weights actually underperformed a random shuffle. Before concluding the model was broken, I ran a noise-ceiling check — comparing two independent Monte Carlo draws of the same reality against each other — which showed ~98% Precision@10 self-agreement, proving the task itself was learnable and the gap was real, not simulator noise. Then I fixed it with a grid search over the weight simplex on a disjoint train/test split.

Q: The grid search put 100% weight on one factor and zeroed out the rest — doesn't that mean your original 4-factor design was pointless?

A: Not exactly — it means that *specific ground-truth metric* (P95 revenue-at-risk from the simulator) is mechanically dominated by revenue exposure, because the simulator's own impact formula multiplies supply share by unit cost and demand — nearly the same terms as the revenue-weighted feature. So the grid search correctly found the best predictor *of that metric*. But geography, reliability, and substitutability likely matter for risk types the simulator doesn't model — correlated regional shocks, reputational damage, second-order supplier failures. I see this as a construct-validity limitation of the simulator's impact formula, not proof the other factors are useless — and it's exactly the kind of thing I'd want to fix by diversifying the ground-truth signal, not by declaring victory.

Q: Why grid search instead of gradient descent or a regression model to learn the weights?

A: The objective — Spearman rank correlation — is piecewise-constant in the weights (small weight changes often don't change any pairwise ranking), so it's non-differentiable almost everywhere, which breaks gradient-based methods. With only 5 factors constrained to the probability simplex, an exhaustive grid at a reasonable resolution (25 steps, ~0.04 granularity) is fully tractable and exact up to that resolution — no need for an approximate optimizer.

Q: What would you improve if you had another week?

A: Three things. First, reconcile the inconsistent threshold/formula schemes I found across the codebase (three different risk-band cutoffs, two reliability formulas) into one source of truth. Second, add caching or a background refresh job — right now almost every API read recomputes the full Monte Carlo or centrality analysis live, which won't scale past a small supplier count. Third, diversify the simulator's ground truth so the evaluation isn't dominated by one feature — e.g., add a correlated-regional-shock scenario or a reputational-risk term that isn't just a function of revenue exposure, so the other three risk factors get a fair chance to prove their value.

Q: How is the synthetic data trustworthy if it's fake?

A: It's synthetic but not arbitrary — I fit real distributions (delay times via Normal/Gamma MLE with KS goodness-of-fit selection, category late-delivery rates, country market shares, demand seasonality, log-normal cost parameters) from the public DataCo Global Supply Chain dataset, then generated new suppliers/products/transactions from those fitted distributions. I also deliberately seeded a few seed known-risk cases — a hardcoded sole-source supplier and a 3-way geographic cluster — specifically so I'd have ground-truth cases to confirm the graph and scoring algorithms actually catch.

Q: Why NetworkX / a bipartite graph representation instead of just aggregating tables?

A: Some risk is inherently structural/topological, not just "how big is this supplier's revenue." A supplier can look small by revenue but be structurally critical if it's the sole path to a high-demand product, or if removing it would fragment the network. Modeling suppliers and products as a bipartite directed graph with PageRank and centrality lets me capture that — degree centrality for breadth of exposure, betweenness for bottleneck/bridge risk, closeness for how fast disruption effects would propagate, PageRank (on the reversed graph) for demand-weighted importance.

Q: What's the ROI number actually measuring — is it a real financial ROI?

A: It's a benefit-cost ratio: dollars of expected risk eliminated per dollar spent on the mitigation action, not IRR or NPV. I compute expected annual loss as P95 exposure times the worst-case scenario's annual probability — which I'll flag as a conservative approximation, since P95 is a tail statistic, not a true expectation, so multiplying it by probability isn't a strict expected-value calculation. It's intentionally conservative rather than precise.

Q: Tell me about a mistake or gap you found in your own project.

A: The composite resilience score initially validated fine using Pearson correlation against simulated exposure in one of my notebooks, but when I built a proper rank-based evaluation harness later, it turned out to rank suppliers worse than random. Pearson can look strong when a few extreme-exposure suppliers dominate the correlation even if the actual ordering of most suppliers is scrambled — Spearman, which only cares about rank order, caught that. It taught me to pick evaluation metrics that match the actual downstream use case (here: ranking suppliers by priority, not predicting exact dollar amounts) rather than whichever metric happens to look best.

22. Formula & Number Cheat Sheet

Concept	Value / Formula
Suppliers / Products	100 / 500
Transaction history	730 days (2 years)
Monte Carlo runs	10,000 per supplier×scenario (5,000,000 total per full batch)
PageRank damping factor α	0.85
PageRank graph direction	Reversed (Product→Supplier)
Resilience score weights	Dependency 0.40 / Geographic 0.25 / Reliability 0.20 / Substitutability 0.15
Dependency risk clip	Max supply share capped at 0.50
Geographic cluster penalty	+0.15 flat if country has ≥ 3 suppliers
Reliability risk (scoring.py)	$0.5 \cdot \text{norm}(\text{delay}) + 0.5 \cdot \text{norm}(\text{volatility})$
Reliability score (ETL notebook)	$1 - (0.4 \cdot \text{delay} + 0.4 \cdot \text{volatility} + 0.2 \cdot \text{rejection})$
Sole-source substitutability	Hard-coded to 1.0
Beta distribution fit	Method of Moments / PERT (span $\approx 6\sigma$), fallback $\alpha=\beta=4.0$
Impact formula	$\text{loss_fraction} \times (\text{duration}/30) \times \Sigma(\text{supply_share} \times \text{unit_cost} \times \text{monthly_demand})$
Port Strike probability / geo boost	0.18 annual, $\times 1.5$ for China/US/Mexico/Brazil
Factory Shutdown probability	0.12 annual, $\times (1 + \text{norm_volatility})$
Currency Shock probability	0.12 annual (country-wide flag unused)
Logistics Delay probability	0.35 annual (highest, no modifier)
Quality Failure probability	0.10 annual, $\times (1 + \text{norm_rejection})$
Dual-sourcing cost / reduction	8% of revenue exposure / 50% dep., 60% sub.
Geographic diversification cost / reduction	12% of revenue exposure / 45% geo.
Supplier development cost / reduction	flat \$200,000 / 40% reliability
Quarterly monitoring cost / reduction	flat \$15,000 / 15% reliability
Grid search resolution	25 steps (~ 0.04 granularity) over 5-factor simplex
Learned weights result	100% revenue_weighted, 0% all others
Composite score eval (Spearman)	≈ -0.047 (worse than random ≈ 0.016)
Noise ceiling (Precision@10)	≈ 0.98 (full noise) / 0.94 (pure sim noise)
Learned weights test performance	Spearman ≈ 0.9993 , Precision@10 ≈ 0.956

Test suite	45 tests, ~1.5s runtime, all passing
------------	--------------------------------------

Compiled from a full read-through of the ShockProof / supply-chain-shock-simulator codebase — for personal interview preparation.